El algoritmo de Dijkstra

Grupo 2

18 de abril de 2026

UNIVERSIDAD DE CÓRDOBA

Integrantes del grupo

Alijo Fernández, Miguel Ángel
Buzón García, Alejandro
Casado Caballero, Antonio
Collantes Aguilar, Pablo
Deogracias Moya, Juan
Guerrero Romero, Jose Francisco
Laredo Gutiérrez, José Raúl
López Reyes, Marcos
Aramyan, David Ángel
Costa Ruiz-Canela, Javier Jesús
Ligero González, Alejandro

UNIVERSIDAD DE CÓRDOBA

Tabla de contenidos

1. Introducción
2. Descripción del algoritmo
3. Fundamento matemático
4. Ejemplo: Desarrollo paso a paso
5. Explicación del pseudocódigo
6. Simulación del algoritmo
7. Conclusiones
8. Bibliografía y webgrafía

UNIVERSIDAD DE CÓRDOBA

Introducción

Descripción del algoritmo

Fundamento matemático

Para comprender el algoritmo de Dijkstra desde una perspectiva rigurosa, es necesario establecer los fundamentos que garantizan su correctitud. Esta sección presenta las definiciones formales y las propiedades estructurales en las que se basa el algoritmo.

Definición (Grafo ponderado)

Un grafo ponderado es un par (G, w), donde $G = (V, E)$ es un grafo y $w : E \to \mathbb{R}^+$ es una función que asigna pesos positivos a las aristas del grafo.

UNIVERSIDAD DE CÓRDOBA

Fundamento matemático: Camino y Distancia

Definición (Camino y su peso)

Dado un grafo ponderado (G, w) y un camino $C : v_0 v_1 \ldots v_k$, se define el peso del camino C como la suma de los pesos de las aristas que lo componen:

$$w(C) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Definición (Distancia entre vértices)

Para dos vértices distintos $u, v \in V$, la distancia $d_G(u, v)$ es:

$$d_G(u, v) = \text{mín}\{w(C) \mid C \text{ es un } u\text{-}v \text{ camino}\}$$

Si no hay camino, $d_G(u, v) = +\infty$.

Principio de optimalidad y Relajación

Principio de optimalidad de Bellman
Si un camino es mínimo, cualquier subcamino contenido en él también es un camino mínimo entre sus propios extremos.

Demostración.

Si existiera un subcamino más corto, el peso total del camino original podría reducirse, lo que contradice que era el mínimo. □

Operación de relajación
El algoritmo mantiene para cada vértice v una estimación de su distancia mínima $d[v]$ desde la fuente s (inicialmente $d[s] = 0$ y $d[v] = +\infty$ para el resto).
Para cada arista $(u, v) \in E$:

$$\text{si } d[u] + w(u, v) < d[v] \implies d[v] \leftarrow d[u] + w(u, v)$$

UNIVERSIDAD DE CÓRDOBA

Condiciones y Complejidad

Condición de pesos no negativos

Requiere que la función de peso sea $w : E \to \mathbb{R}^+$. Pesos negativos invalidarían la estrategia *greedy*, ya que una distancia "finalizada" podría reducirse posteriormente por un arco negativo.

Complejidad computacional

Estructura de datos	**Complejidad temporal**
Lista o Arreglo simple	$O(\|V\|^2)$
Heap binario	$O((\|V\| + \|E\|) \log \|V\|)$
Heap de Fibonacci	$O(\|E\| + \|V\| \log \|V\|)$

UNIVERSIDAD DE CÓRDOBA

Ejemplo: Desarrollo paso a paso

Pseudocódigo del algoritmo

```
function Dijkstra(Graph, source):
   dist[source] = 0
   for each vertex v in Graph:
       if v != source:
           dist[v] = infinity
       add v to Q (unvisited set)
   while Q is not empty:
       u = vertex in Q with min dist[u]
       remove u from Q
       for each neighbor v of u in Q:
           alt = dist[u] + length(u, v)
           if alt < dist[v]:
               dist[v] = alt
               prev[v] = u
   return dist, prev
```

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo

El algoritmo de Dijkstra calcula la ruta más corta desde un nodo inicial hacia todos los demás nodos del grafo. Requiere que todas las distancias sean positivas o cero. Su idea principal es seleccionar siempre el nodo no visitado más cercano al origen y actualizar las distancias de sus nodos vecinos si encuentra un camino más corto pasando por él.

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo: línea por línea I

- `dist[source] = 0`: La distancia para ir del punto de inicio a sí mismo es cero.
- `for each vertex v in Graph`: Recorre todos los demás puntos o nodos de la red.
- `dist[v] = infinity`: Como al principio el algoritmo no sabe cómo llegar al resto de nodos, les asigna temporalmente una distancia infinita.
- `add v to Q`: Introduce todos los nodos en un conjunto llamado `Q`, que funciona como una lista de tareas de nodos no visitados.

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo: línea por línea II

- `while Q is not empty`: El proceso se repite en bucle hasta que la lista de nodos no visitados quede vacía.
- `u = vertex in Q with min dist[u]`: Selecciona de la lista el nodo que tenga la menor distancia acumulada. En la primera iteración, este será necesariamente el nodo origen.
- `remove u from Q`: Elimina ese nodo de la lista para marcarlo oficialmente como visitado.
- `for each neighbor v of u in Q`: Revisa únicamente los nodos vecinos de `u` que todavía no hayan sido visitados.

UNIVERSIDAD DE CÓRDOBA

Explicación del pseudocódigo: línea por línea III

- `alt = dist[u] + length(u, v)`: Calcula la longitud de una posible nueva ruta hasta `v`, sumando la distancia ya conocida hasta `u` más el peso de la arista entre `u` y `v`.
- `if alt < dist[v]`: Este es el núcleo del algoritmo. Comprueba si la nueva distancia calculada es menor que la mejor distancia conocida hasta ese momento para llegar a `v`.
- `dist[v] = alt` y `prev[v] = u`: Si la nueva ruta es mejor, se actualiza la distancia mínima registrada y se guarda que el nodo anterior óptimo para llegar a `v` es `u`.

UNIVERSIDAD DE CÓRDOBA

Resultado final del algoritmo

- `return dist, prev`: El algoritmo finaliza devolviendo dos estructuras:
  - `dist`, que contiene la distancia mínima desde el nodo origen hasta cada nodo del grafo.
  - `prev`, que permite reconstruir el camino más corto seguido para llegar a cada nodo.

Gracias a estas dos estructuras, no solo se conoce cuánto cuesta llegar a cada punto, sino también cuál es la ruta exacta que debe seguirse.

UNIVERSIDAD DE CÓRDOBA

Simulación del algoritmo

En el siguiente enlace encontrarás una simulación **desarrollada por nosotros** del algoritmo programa en `react.js`:

https://djkistra.vercel.app/

UNIVERSIDAD DE CÓRDOBA

Conclusiones

Bibliografía y webgrafía

- Insertar fuentes.